



QATIP Intermediate AWS Lab 03

Setting Up AWS Secrets Manager and IAM user for Terraform Authentication

Contents

Lab Objectives	2
Teaching Points	2
Step 1: Administrator Tasks	2
Step 2: Developer Tasks	2
Before you begin	2
Administrators Set Up Secrets Manager and IAM User	2
Lab requirement.....	2
Administration creates an IAM User and Secret.....	3
Developers Retrieve IAM Credentials from AWS Secrets Manager	3
Define Parameters and Secure Retrieval Function.....	3
Verify the Active AWS Identity & Run Terraform.....	3
Using IAM Roles with Terraform	4
Revert to the Lab Identity (student)	5
Create the IAM Role	5
Inspect the IAM Role	5
List the permission policies attached to the role	6
Inspect the permission policy	6
Configure Terraform to Assume the Role.....	7
Lab clean-up	8

Lab Objectives

This lab presents methods by which multiple developers can authenticate Terraform dynamically without needing to store or share sensitive credentials directly.

Teaching Points

Step 1: Administrator Tasks

- The security team sets up **AWS Secrets Manager** to securely store Terraform authentication credentials.
- An **IAM User** is created with restricted access.

Step 2: Developer Tasks

- Developers retrieve authentication credentials using **AWS Secrets Manager**.
- Developers can **retrieve** credentials but **cannot modify** them.

Before you begin

1. In the IDE terminal pane, enter the following command...
cd ~/environment/aws-tf-int/labs/03
2. This shifts your current working directory to labs/03. Ensure all commands are executed in this directory
3. Close any open files and use the Explorer pane to navigate to and open the labs/03 folder.

Administrators Set Up Secrets Manager and IAM User

Lab requirement

4. Update the Cloud9 environment to install **jq**, a lightweight command-line tool for parsing, filtering, and transforming JSON data...

sudo apt-get update && sudo apt-get install -y jq

Administration creates an IAM User and Secret

5. Examine the script **setup-terraform-secrets.sh** in **aws-tf-int/labs/03**
6. Ensure lab shell scripts are executable...

```
chmod +x setup-terraform-secrets.sh
chmod +x retrieve-aws-credentials.sh
```

7. Execute the script: **./setup-terraform-secrets.sh**
8. To verify the secret is created you can run
**aws secretsmanager get-secret-value **
**--region us-east-1 **
--secret-id terraform-aws-credentials

Developers Retrieve IAM Credentials from AWS Secrets Manager

Developers retrieve stored credentials using AWS CLI and export them as environment variables.

Define Parameters and Secure Retrieval Function

9. Examine the script **retrieve-aws-credentials.sh** in **aws-tf-int/labs/03**
10. Execute the script: **./retrieve-aws-credentials.sh**
11. Check stored credentials: **cat aws_credentials.env**
12. Load the stored credentials into your session:
source aws_credentials.env
13. Verify that you are now using the terraform-user credentials by running: **aws sts get-caller-identity**

```
awsstudent:~/environment/aws-tf-int/labs/03 (main) $ aws sts get-caller-identity
{
  "UserId": "AIDAJL4H76:use/terraform-user",
  "Account": "8876",
  "Arn": "arn:aws:iam::8876:use/terraform-user"
}
```

Verify the Active AWS Identity & Run Terraform

Developers now use Terraform with the retrieved AWS credentials.

14. Update line 18 of **main.tf** with a unique S3 bucket name of your choice and save the change.
15. Ensure AWS credentials are loaded: **source aws_credentials.env**
16. Initialize Terraform: **terraform init**
17. Test infrastructure deployment: **terraform plan**
18. Deploy the infrastructure: **terraform apply**
19. Verify the AWS resources in the AWS Console.

Using IAM Roles with Terraform

The previous exercise demonstrated how Terraform can authenticate using long-lived IAM user credentials retrieved securely from AWS Secrets Manager.

Whilst this approach is suitable for many scenarios, AWS best practice is to minimise the use of long-lived credentials. Instead, users authenticate using their normal AWS identity and are granted permission to **assume an IAM Role** containing the permissions required to manage infrastructure.

When Terraform is configured to use an IAM Role, it initially authenticates using the developer's existing AWS credentials. The AWS provider then requests temporary credentials for the specified IAM Role using AWS Security Token Service (STS). Once granted, Terraform performs all subsequent AWS operations using the temporary role credentials rather than the developer's original identity.

In a production environment:

- Cloud administrators create and manage IAM Roles.
- Terraform developers are granted permission to assume those roles.
- Developers do not create or manage the roles themselves.

For the purposes of this lab, you will perform both administrative and developer tasks.

Revert to the Lab Identity (student)

21. Before demonstrating role assumption, ensure you are no longer using the temporary terraform-user credentials retrieved from Secrets Manager.

22. Remove the exported environment variables:

```
unset AWS_ACCESS_KEY_ID
unset AWS_SECRET_ACCESS_KEY
unset AWS_SESSION_TOKEN
```

23. Confirm that your active identity has reverted to the default laboratory user:

```
aws sts get-caller-identity
```

The returned ARN should identify the student user.

Create the IAM Role

The platform team has pre-created the permissions policy **terraform-user-policy**. You will create an IAM Role named **TerraformRole**, define who may assume it using **trust-policy.json**, and then attach the existing permissions policy to the role

24. Create the role and attach the supplied policy:

```
aws iam create-role --role-name TerraformRole \
--assume-role-policy-document file://trust-policy.json \
--no-cli-pager

aws iam attach-role-policy \
--role-name TerraformRole \
--policy-arn arn:aws:iam::$(aws sts get-caller-identity --query
Account --output text):policy/terraform-user-policy
```

Inspect the IAM Role

Before configuring Terraform to assume the role, let's examine the role that has been created. This helps illustrate the two fundamental components of every IAM Role:

- **Trust Relationship** – Who is permitted to assume the role?

- **Permission Policies** – What actions may the role perform once it has been assumed?

25. Inspect the role by running the following command:

```
aws iam get-role --role-name TerraformRole --no-cli-pager
```

Review the output and locate the **AssumeRolePolicyDocument**.

This is the role's **trust relationship**, which defines the identities that are permitted to assume the role.

The trust policy does **not** grant permissions to AWS resources. It simply answers the question: **Who may become this role?**

List the permission policies attached to the role

26. Run:

```
aws iam list-attached-role-policies --role-name TerraformRole --no-cli-pager
```

You should see the **terraform-user-policy** attached to the role.

Permission policies are separate from the trust relationship.

The trust relationship determines **who may assume the role**.

The attached permission policies determine **what the role may do once assumed**.

Inspect the permission policy

27. Replace <ACCOUNT_ID> with your AWS account ID and execute:

```
aws iam get-policy-version --policy-arn  
arn:aws:iam::<ACCOUNT_ID>:policy/terraform-user-policy --  
version-id v1 --no-cli-pager
```

Review the actions granted by the policy.

These permissions become available **only after** the role has been successfully assumed.

An IAM Role consists of two independent parts:

- **Trust Relationship** – Who may assume the role?
- **Permission Policies** – What actions the role may perform once assumed.

Both are required. A role that trusts nobody can never be assumed. A role with no permission policies can be assumed successfully, but cannot perform any useful work.

Configure Terraform to Assume the Role

28. Uncomment all commented lines in **main.tf**.
29. Replace <Account_ID> with your AWS account ID.
30. Reinitialise Terraform:

terraform init -upgrade

31. Execute:

terraform plan

terraform apply --auto-approve

Terraform is still launched using your aws student credentials. However, because the AWS provider is configured with an **assume_role** block, it requests temporary credentials for TerraformRole before performing any AWS operations.

32. The plan should report **no infrastructure changes**, as the bucket already exists.
33. Modify the bucket name and repeat the **plan/apply**.
34. Observe the output:

terraform_identity = arn:aws:sts::<account-id>:assumed-role/**TerraformRole**/...

Notice that Terraform is **no longer executing as student**.

The original user authenticated to AWS and requested permission to assume the role. AWS then issued temporary credentials representing TerraformRole, and Terraform used those credentials to perform all subsequent operations.

Lab clean-up

35. Run **terraform destroy --auto-approve**

Congratulations, you have completed this lab

Copyright

© 2026 QA Michael Coulling-Green. All rights reserved. These lab materials are provided as part of an authorised training course. Course attendees may reuse these materials for their own personal learning and reference outside of the training session.

Unauthorised copying, redistribution, publication, or use of these materials to deliver training is prohibited without prior written permission from the author.

© QA 2026